GSPS Automation Implementation Brief

Repository discovery, implementation requirements, and Claude Code instructions

Prepared from the current session. Scope: Guided Stock Projection System (GSPS) scan-to-plan automation, paper/live modes, Wall Street member automation, evidence testing, and live-only risk protections.

Executive directive

Implement a closed-loop GSPS workflow in which a qualifying scan signal automatically creates one durable candidate trade plan. A Wall Street-level member may then activate a separate automation profile from the restricted Automation tab in either paper or live mode. The scan itself never auto-activates live execution.

The Seven Hermetic Principles, as discussed in Dennis William Hauck's *The Emerald Tablet: Alchemy for Personal Transformation*, are the product-design vocabulary and operating philosophy. They are not a scientific claim that Hermetic concepts predict markets. Every trading rule, parameter, and performance result must remain explicit, versioned, and empirically testable.

First task: inspect repo and PR

Claude Code must first find the relevant repository, current branch, and pull request; inspect the PR body for the two open follow-ups; then produce an evidence-based gap report before editing code.

```
You are working in the current Git repository. Before implementing anything, identify the repository, current branch, and the pul

1. Run:
   git remote -v
   git branch --show-current
   git status --short
   git log --oneline -10

2. If GitHub CLI is available and authenticated, run:
   gh repo view --json nameWithOwner,url,defaultBranchRef
   gh pr view --json number,title,url,body,state,baseRefName,headRefName,files,commits

3. If `gh pr view` cannot identify a PR, inspect open PRs for the current head branch:
   gh pr list --head "$(git branch --show-current)" --state open --json number,title,url,body,headRefName,baseRefName

4. Search the repository and PR body/comments for these exact concepts and close variants:
   - "no UI"
   - "auto-creates a plan"
   - "qualifying signal"
   - "scan pipeline"
   - "trade plan"
   - "automation"
   - "paper trading"
   - "backtest" and "forward test"
   - "Wall Street"

5. Inspect code and database migrations for existing implementations. Use ripgrep, then open every relevant file:
   rg -n -i "qualif|signal|scan.*pipeline|trade.*plan|automation|paper.*trad|live.*trad|backtest|forward.*test|broker|order|posit

6. Read package.json, database migration files, API/server-action routes, job/queue/cron configuration, authentication/role check

7. Produce a concise gap report before edits with these headings:
   - Repository and PR identified
   - Existing signal-to-plan flow
   - Existing schema and migrations
   - Existing Automation UI/API
   - Existing paper/live separation
   - Existing broker, notification, and school integrations
   - Exact missing files and proposed files to change
   - Risks, assumptions, and questions that truly block implementation

Do not deploy, merge, delete data, create production orders, change production secrets, or add real broker credentials.
```

System intent

Layer	Required behavior	Authority
Signal	A deterministic, versioned GSPS rule evaluates a market condition.	Scan engine
Candidate plan	Every qualifying signal automatically produces one idempotent, durable GSPS candidate plan.	Scan engine
Automation profile	A member deliberately elects to automate an eligible plan in paper or live mode.	Member + server validation
Execution	Paper engine simulates. Live broker adapter can submit orders only after server-side authorization.	Server-side automation engine
Evidence loop	Historical replay/backtest and timestamped forward testing use the same signal and plan-generation logic.	Testing and reporting

Hermetic design mapping

Principle	GSPS system expression
Mentalism	Strategy rules and parameters are explicit, immutable by version, and precede discretionary interpretation.
Correspondence	Capture multi-timeframe, ticker/sector/index, and market-regime context with signal provenance.
Vibration	Model measurable momentum, volatility, volume, price movement, and data freshness.
Polarity	Every bullish thesis has an explicit invalidation and opposite-direction pivot path; no direction reversal without a qualifying
Rhythm	Analyze time/session/cycle and regime features only through testable rules and performance breakdowns.
Cause and Effect	Persist the full signal → plan → order/simulation → outcome chain with immutable identifiers.
Gender	Generate hypotheses/plans, then constrain selection and execution through risk, validation, and evidence gates.

Core workflow

```
Market data / historical replay
  → scan run
  → normalized signal
  → qualification under frozen strategy version
  → qualifying signal persisted
  → candidate plan auto-created once (idempotent)
  → eligible-for-automation
  → member selects paper or live in Automation tab
  → server validates permitted configuration
  → automation profile activated
  → entry condition occurs
  → paper simulation OR live broker order
  → order/fill/position lifecycle events
  → backtest and forward-test evidence reporting
```

Non-qualifying signals must never create plans. A qualifying signal must create one candidate plan automatically; it must not automatically place a live order or activate a member automation profile.

Required data model

• Use or introduce: scan_runs, signals, strategy_versions, trade_plans, plan_events, automation_profiles, automation_events, order_intents, broker_orders, broker_fills, positions, risk_limits, notification contacts/delivery verification, trading_access_restrictions, and school completion records.

• Link trade_plans to source_signal_id, source_scan_run_id, strategy_version, and a stable signal fingerprint.

• Store the price/timeframe/data timestamp, decimal-strip calculation metadata, digit sum, modulo-9 root, 3/6/9 harmonic vectors, entry, TP1, master target, stop/invalidation, polarity pivot, plan status, source, and audit timestamps.

• Store all financial amounts as fixed decimal/numeric values, never JavaScript floating-point values.

• Use event tables for audit history; do not overwrite original plan conditions after their outcomes are known.

Candidate plan rules

```
function candidatePlanKey(signal, strategy) {
  return [signal.ticker, signal.timeframe, strategy.version, signal.fingerprint].join(':');
}
```

```
if (signal.qualifies) {
  createOrUpsertCandidatePlan({
    sourceSignalId: signal.id,
    sourceScanRunId: scanRun.id,
    strategyVersion: strategy.version,
    idempotencyKey: candidatePlanKey(signal, strategy),
    generatedBy: 'gsps_scan_pipeline',
  });
}
```

• Enforce idempotency with a database unique index or constraint on the deterministic plan key.

• Create PLAN_AUTO_CREATED_FROM_QUALIFYING_SIGNAL in the audit/event log.

• Define behavior for materially changed signals. Prefer: create a new plan version and mark an unreviewed/active prior plan superseded; never mutate an historic plan's original thesis invisibly.

• Use the same plan generator in scans, backtesting, and forward testing so the research result matches operational behavior.

Automation tab

Create or complete a Wall Street-member-only Automation tab. Authorization must be enforced in the UI, API/server actions, and database Row Level Security (RLS) or equivalent policy. Hiding a tab is not authorization.

Mode	Behavior
System-plan automation	Member selects an eligible system-generated plan. Entry, targets, stop/invalidation, and polarity behavior remain wi
Guided custom automation	Member filters/selects eligible GSPS plans and may configure only schema-approved settings: approved assets/tim
Paper	Simulated executions and simulated positions only. No broker calls and no live-risk consequences.
Live	Broker execution is allowed only inside this tab and only through server-side plan eligibility and pre-trade risk gates.

Live-only policy

The following rules apply exclusively when execution_mode is **live**. They do not apply to paper trading. An active automation profile's mode must be immutable; a live position may not be converted to paper to avoid controls.

Live event	Required behavior
Default stop	Use the GSPS-calculated stop/invalidation coordinate.
Widen/remove stop	Wall Street user may widen or remove it after a high-friction warning, verified email delivery, verified phone/SMS deli
6%, 9%, 15% loss	Send email and phone/SMS notifications once per threshold per trade.
30% loss	Issue a hard warning regardless of user level.
50% loss	Pause the profile; cancel pending orders; submit/retry/reconcile a position close; restrict live trading; keep paper trad

The loss calculation is based on total funds allocated to that individual trade. Define and test treatment for realized partial exits, current liquidation value, fees, and slippage. The system must record close attempts and residual exposure because a broker cannot guarantee an exact exit price in every market condition.

Pre-trade authorization

```
authorizeAutomatedOrder(profileId) must resolve all order terms server-side.

Allow only when:
- profile.execution_mode === 'live' or 'paper'
- user has the required Wall Street entitlement
- plan is eligible, unexpired, unsuperseded, and not invalidated
- strategy version is approved
- market data is fresh
- requested/member configuration validates against the plan's GSPS configuration schema
- risk limits permit the trade
- no kill switch or live-trading restriction applies
- the entry is within its permitted price/time window
- duplicate order protection passes
- for live only: broker connection is valid, and stop-override/contact requirements pass where applicable
```

```
The client must send an automation_profile_id, not arbitrary ticker, side, size, stop, or price values.
```

UI requirements

• Scanner/Trade Plans: scan health, last successful run, freshness/errors, qualifying signals, active/candidate plans, invalidated plans, and plan history.

• Plan grid: ticker, direction, current price, Gann root, entry, TP1, master target, stop, timeframe, status, plan age, and source timestamp.

• Plan detail: full Gann Coordinate Execution Matrix, decimal-strip inputs, qualification rationale, polarity pivot, strategy version, source signal, scan run, and complete event history.

• Automation: eligible-plan list, clear block reasons, mode selection, bounded configuration form, risk preview, activation flow, personal and plan-level pause/stop controls, active automation state, and broker/order audit trail.

• Risk center: exposure, daily risk capacity, positions, P&L;, data/broker health, and kill-switch state.

• Label all auto-created plans “System-generated candidate — not an execution instruction.”

Backtest and forward test

• Backtesting must use the identical versioned qualification and plan-generation engine, historical inputs available at decision time, and conservative fill assumptions. Guard against look-ahead bias, survivorship bias, and duplicate-signal inflation.

• Persist a backtest run’s strategy version, parameters, date range, data source/version, modeled costs/slippage, trade-level plans, entries/exits, and metrics.

• Forward testing must generate timestamped plans before their outcome is known. Do not rewrite entry/stop/target after observing the market. Changes create a new plan/version.

• Report expectancy, win rate, average win/loss, profit factor, maximum drawdown, count, time in trade, and breakdowns by strategy version, symbol, timeframe, direction, root/vector, session, and market regime.

Implementation sequence

• Inspect repository/PR and write the gap report.

• Preserve existing qualification behavior; formalize strategy-version and signal schemas if absent.

• Create migrations and RLS for durable signals, plans, lifecycle events, automation profiles, execution records, and risk/school state.

• Implement idempotent qualifying-signal → candidate-plan generation.

• Implement plan eligibility, bounded configuration schema validation, automation profile lifecycle, and event ledger.

• Implement paper simulation first or maintain strict simulation isolation where it already exists.

• Implement live broker adapter behind server-side pre-trade gates, reconciliation, duplicate protection, and kill switches.

• Implement the live-only stop override and loss-threshold monitor/notification/forced-close/restriction workflow.

• Implement scanner, plan detail, Automation, and risk-center UI.

• Build historical replay/backtesting and forward-test reporting using the same plan engine.

• Add tests, run checks, and update the PR body. Do not merge or deploy.

Acceptance tests

• A non-qualifying signal creates no candidate plan.

• A qualifying signal creates exactly one candidate plan; reruns with the same fingerprint do not duplicate it.

• Every candidate plan retains source signal, scan run, strategy version, calculations, coordinates, polarity/pivot, and audit events.

• An invalidated, expired, stale, superseded, or otherwise ineligible plan cannot be automated.

• Only Wall Street members can create/activate live automation profiles, with server-side authorization tested.

• Configuration outside the GSPS-approved schema is rejected server-side.

• Paper profiles create only simulated orders/positions and never call brokers, trigger mandatory live notices, force-close, restrict live access, or require school re-completion.

• Live stop widening/removal is blocked unless entitlement, warning acknowledgement, verified email, verified phone/SMS, and audit events are present.

• Live losses send 6%, 9%, 15%, and 30% alerts once each per trade; no duplicate alerts on repeated monitor jobs.

• At a live 50% allocated-funds loss, the system pauses automation, cancels pending orders, attempts/reconciles closure, records all events, restricts live trading, and leaves paper access available.

• A restricted user cannot reactivate live trading until the required GSPS School completion/re-completion is verified server-side.

• UI has loading, empty, blocked, error, and success states and shows the server-backed audit trail.

• Lint, typecheck, test suite, build, and database migration validation pass.

Claude Code implementation prompt

```
Implement the GSPS requirements in this repository, beginning with the repository/PR discovery and gap report described earlier i

Core requirement: when a GSPS scan signal qualifies under a frozen strategy version, automatically persist exactly one idempotent

Create a restricted Wall Street-member Automation tab where a member can deliberately create an automation profile from an eligib

Paper mode: use simulation only. Never call live broker adapters, issue mandatory live-risk alerts, force close, restrict live ac

Live mode only: a Wall Street-level member may retain, widen, or remove a plan’s stop. Widen/remove requires a high-friction warn

Implement and test the associated data model, migrations, RLS/authorization, candidate-plan service, event ledger, Automation API

Use the same versioned signal/plan engine for scan operation, backtests, and forward tests. Add a backtest/forward-test foundatio

Before changing files, show the discovery/gap report. Then implement in small reviewable commits or a clearly structured diff. Ru
```

Non-negotiable constraints

• Do not use Hermetic framing as evidence of market predictability; preserve empirical validation, reproducibility, and clear risk disclosure.

• Do not permit client-side bypass of GSPS execution constraints.

• Do not implement arbitrary order-entry endpoints that accept user-supplied ticker, side, price, stop, or quantity without server-side derivation and validation.

• Do not expose or create real broker credentials in source control.

• Do not merge, deploy, or submit real orders as part of this implementation task.

• Do not apply live-only loss enforcement to paper trading.